

1. Objectives:

1. To understand the concept of stack and its operations such as push, pop, and peek using array implementation.
2. To apply stack in expression handling by converting infix expression to postfix form using operator precedence.

2. Introduction:

- **Stack:** Stack is a linear data structure that follows the Last In First Out (LIFO) principle, meaning the element inserted last is removed first. It can be implemented using arrays or linked lists. The primary operations of a stack are **push** (inserting an element), **pop** (removing the top element), and **peek** (viewing the top element without removing it). In array implementation, a variable called **top** is used to keep track of the current position of the stack. Initially, **top** is set to -1 indicating that the stack is empty.

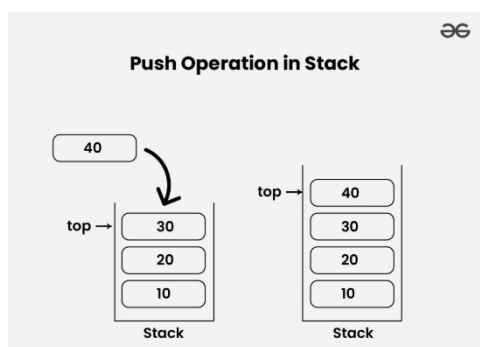


Fig 8.1: Stack Push

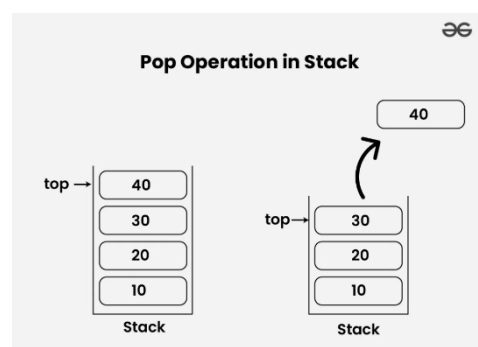


Fig 8.2: Stack Pop

In the push operation, a new element is inserted into the stack by first checking whether the stack is full (i.e., $\text{top} == \text{size} - 1$). If the stack is not full, the value of **top** is increased by 1 and the new element is stored at $A[\text{top}]$. In the pop operation, the top element is removed by decreasing the value of **top** by 1 after checking that the stack is not empty (i.e., $\text{top} == -1$). The peek operation is used to display the top element of the stack without removing it.

- **Infix to Postfix:** Operands are directly added to the output string, while operators are pushed into the stack based on their precedence. When an operator with lower or equal precedence is encountered, elements from the stack are popped and added to the postfix expression until the condition is satisfied. Parentheses are also handled using the stack by pushing opening brackets and popping elements until the matching bracket is found.

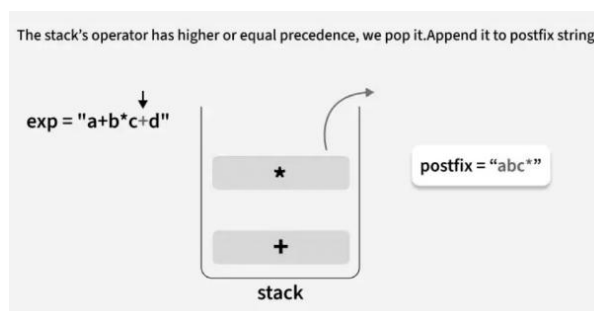


Fig 8.3: Infix to Postfix

3. Procedure

Experiment 1: Push, Pop, Peek

Input:

- k – Element to be inserted into stack
- mx – Maximum size of stack
- A[] – Array used as stack
- top – Index of top element

Output:

- “Stack Full” if overflow
- “Stack Empty” if underflow
- Top element of stack

Algorithm:

StackPush(top, mx, A[]):

1. input k
2. **if** top == mx - 1 **then**
3. print "Stack Full"
4. **else**
5. top $\leftarrow$ top + 1
6. A[top] $\leftarrow$ k
7. **return** top

StackPeek(top, A[]):

1. **if** top == -1 **then**
2. print "Stack Empty"
3. **else**
4. print "Stack Top: ", A[top]

StackPop(top, A[]):

1. **if** top == -1 **then**
2. print "Stack is Empty"
3. **else**
4. top $\leftarrow$ top - 1
5. **return** top

Experiment 2: String Reverse

Input:

- st – Array stack
- A – Given string
- n – Size of string

Output:

- Printed reversed string

Algorithm:

StringReverse(st, A, n):

1. top $\leftarrow$ -1
2. **for** i $\leftarrow$ 0 to n-1 **do**
3. top $\leftarrow$ top + 1
4. st[top] $\leftarrow$ Arr[i]
5. **while** top $\neq$ -1 **do**

6. print st[top]
7. top $\leftarrow$ top - 1
8. end

Experiment 3: Infix to Postfix

Input:

- eq – Infix expression (string)

Output:

- postfix – Converted postfix expression

Algorithm:

Precedence Function prec(c):

1. if c == '/' then return 5
2. else if c == '*' then return 4
3. else if c == '+' then return 3
4. else if c == '-' then return 2
5. else return 1

InfixToPostfix(eq):

1. input eq
2. postfix $\leftarrow$ ""
3. create empty stack st
4. for i $\leftarrow$ 0 to length(eq) - 1 do
5. if eq[i] is operand then
6. postfix $\leftarrow$ postfix + eq[i]
7. else if eq[i] == '(' then
8. push eq[i] into st
9. else if eq[i] == ')' then
10. while st.top() != '(' do
11. postfix $\leftarrow$ postfix + st.top()
12. pop st
13. pop st
14. else
15. while st not empty AND prec(st.top()) $\geq$ prec(eq[i]) do
16. postfix $\leftarrow$ postfix + st.top()
17. pop st
18. push eq[i] into st
19. while st not empty do
20. postfix $\leftarrow$ postfix + st.top()
21. pop st
22. print postfix

4. Required Hardware and Software:

- **Hardware:** AMD RYZEN 7, RAM 8 GB
- **Software:** Visual Studio Code 1.109

5. Experiment Implementation:

Experiment 1: Push, Pop, Peek

```
#include <bits/stdc++.h>
using namespace std;

int stackPush(int top, int mx, int A[]){

    int k;
    cin >> k;

    if (top == mx - 1){
        cout << "Stack Full" << endl;
    }
    else{
        top = top + 1;
        A[top] = k;
    }
    return top;
}

void stackPeek(int top, int A[]){
    if (top == -1){
        cout << "Stack Empty" << endl;
    }
    else{
        cout << "Stack Top: " << A[top] << endl;
    }
}

int stackPop(int top, int A[]){
    if (top == -1){
        cout << "Stack is Empty" << endl;
    }
    else{
        top = top - 1;
    }
    return top;
}

int main(){
    int size;
    cout << "Enter Size: ";
    cin >> size;
    cout << "Enter Numbers: ";

    int st[size];
    int top = -1;

    top = stackPush(top, size, st);
    top = stackPush(top, size, st);
    top = stackPush(top, size, st);
    top = stackPush(top, size, st);
    top = stackPush(top, size, st);

    stackPeek(top, st);

    top = stackPop(top, st);
    top = stackPop(top, st);
    top = stackPop(top, st);

    stackPeek(top, st);
    return 0;
}
```

Table 8.1: Experiment 1 output

Input	Expected Output	Actual Output
Enter Size: 5 Enter Numbers: 1 2 3 4 5	Enter Size: 5 Enter Numbers: 1 2 3 4 5 Stack Top: 5 Stack Top: 2	Enter Size: 5 Enter Numbers: 1 2 3 4 5 Stack Top: 5 Stack Top: 2

Experiment 2: String Reverse

```
#include <bits/stdc++.h>
using namespace std;

int main(){
    string A;
    cout << "Enter a string: ";
    cin >> A;

    int n = A.length();
    char st[n];
    int top = -1;

    for (int i = 0; i < n; i++){
        top = top + 1;
        st[top] = A[i];
    }

    cout << "Reversed: ";
    while (top != -1){
        cout << st[top];
        top = top - 1;
    }

    cout << endl;
    return 0;
}
```

Table 8.2: Experiment 2 output

Input	Expected Output	Actual Output
Enter a string: Paris	Enter a string: Paris Reversed: siraP	Enter a string: Paris Reversed: siraP

Experiment 3: Infix to Postfix

```
#include <bits/stdc++.h>
using namespace std;

int prec(char c){
    if (c == '/'){
        return 5;
    }
    else if (c == '*'){
        return 4;
    }
    else if (c == '+'){
        return 3;
    }
    else if (c == '-'){
        return 2;
    }
    else{
        return 1;
    }
}

int main(){
    cout << "Enter Function: ";

    string eq;
    cin >> eq;

    string postfix = "";
    stack<char> st;

    for (int i = 0; i < eq.length(); i++){
        if (isdigit(eq[i])){
            postfix = postfix + eq[i];
        }
        else if (eq[i] == '('){
```

```

        st.push(eq[i]);
    }
    else if (eq[i] == ')'){
        while (st.top() != '('){
            postfix = postfix + st.top();
            st.pop();
        }
        st.pop();
    }
    else{
        while (!st.empty() && prec(st.top()) >= prec(eq[i])){
            postfix = postfix + st.top();
            st.pop();
        }
        st.push(eq[i]);
    }
}

while (!st.empty()){
    postfix = postfix + st.top();
    st.pop();
}

cout << "Postfix: " << postfix;

return 0;
}

```

Table 8.3: Experiment 3 output

Input	Expected Output	Actual Output
Enter Function: A+(12*B)-2/F Postfix: A12B*+2F/-	Enter Function: A+(12*B)-2/F Postfix: A12B*+2F/-	Enter Function: A+(12*B)-2/F Postfix: A12B*+2F/-

6. Discussion & Conclusion:

In this lab experiment, we successfully implemented stack operations using an array, including push, pop, and peek, and observed how overflow and underflow conditions. We also used stack in converting infix expressions to postfix form and reversing a string. This explains the LIFO nature of stack. Some challenges were faced in managing the top pointer correctly and handling operator precedence in expression conversion. Logical errors in stack conditions were more hard than syntax errors and required careful tracing. Overall, this experiment helped to build a clear understanding of stack operations and their practical applications

7. Reference:

1. <https://www.geeksforgeeks.org/cpp/stack-in-cpp-stl/> accessed at 6.25 PM on 18th April 2026.
2. <https://www.geeksforgeeks.org/dsa/convert-infix-expression-to-postfix-expression/> accessed at 12:35 Pm on 24th April